



Präsenzaufgabe 1 [Geldbeträge]

Schreiben Sie die Klasse **Betrag**.

Die Klasse besitzt als Objektattribute die **int**-Attribute **euro** und **cent**.

Die Klasse besitzt den Konstruktor **Betrag(int euro, int cent)**. Bei Aufruf des Konstruktors wird den Objektattributen **euro** und **cent** die Werte der übergebenen lokalen, gleichnamigen Parametern zugewiesen. Das passiert mit Hilfe der vorhandenen Setter. **Halten Sie sich hier an die Reihenfolge: erst euro, dann cent, siehe auch die Beispiele unten.**

Die Klasse besitzt folgende Methoden:

- Die Getter und Setter zu allen Objektattributen. Lassen Sie die Getter und Setter automatisch generieren, um sich an die Bezeichnungskonventionen (insbesondere bei boolean) zu halten.
- Der Setter von **euro** wird anschließend nicht mehr verändert.
- Passen Sie weiterhin den Setter von **cent** an: Der Setter von **cent** überprüft, ob die übergebene Zahl so groß ist, dass ganze Euro daraus resultieren könnten. Ist das der Fall, so wird der Wert von **euro** um den entsprechenden Wert erhöht, und die Anzahl, die in **cent** gespeichert wird, um den entsprechenden Wert gesenkt.

Beispiele:

- Aufruf **setCent(346)**: **euro**-Wert erhöht sich um 3, **cent** wird 46 zugewiesen.
- Aufruf **setCent(6)**: **euro**-Wert erhöht sich um 0, **cent** wird 6 zugewiesen.
- Aufruf **setCent(200)**: **euro**-Wert erhöht sich um 2, **cent** wird 0 zugewiesen.

Beispiele:

- Wird in der **main**-Methode **Betrag(10,45)** aufgerufen, so wird ein Objekt erzeugt, bei dem das Attribut **euro** den Wert 10 und das Attribut **cent** den Wert 45 speichert.
- Wird in der **main**-Methode **Betrag(1,101)** aufgerufen, so wird ein Objekt erzeugt, bei dem das Attribut **euro** den Wert 2 und das Attribut **cent** den Wert 1 speichert.

Insbesondere heißt das: Sind die Werte von **cent** und **euro** schon gesetzt, wird durch **setCent** - falls der übergebene Wert dies erfordert - der in **euro** bereits gespeicherte Wert um den entsprechenden Wert erhöht und was übrig bleibt, wird in **cent** gespeichert. Was also vorher in **cent** gespeichert ist, spielt keine Rolle - es wird überschrieben.

Schreiben Sie in der **main**-Klasse die Methode **summe** nach folgenden Vorgaben.

- Modifikatoren: **public static**
- Rückgabotyp: **Betrag**

- Name: **summe**
- Übergabeparameter: **Betrag a, Betrag b**

Wird **summe(Betrag a, Betrag b)** aufgerufen, so wird ein neues Objekt vom Typ **Betrag** erstellt. Der Objektvariablen **euro** wird die Summe der beiden **euro**-Werte aus **a** und **b** zugewiesen, der Objektvariablen **cent** wird die Summe der beiden **cent**-Wert aus **a** und **b** zugewiesen. Dies geschieht mit Hilfe der vorhandenen Setter.

- Beispiel: **a** steht für den Betrag 13,60€ und **b** für den Betrag 14,80€. Dann steht das Objekt, das von **summe** zurück gegeben wird, für den Betrag 28,40€. Insbesondere ist in **euro** der Wert 28 und in **cent** der Wert 40 gespeichert.

Präsenzaufgabe 2 [Interfaces]

Ein **Interface** legt fest, *welche Methoden* eine Klasse besitzen muss. Es enthält nur Methodenköpfe (keine Implementierungen) und wird mit dem Schlüsselwort **interface** definiert.

Schreiben Sie ein Interface namens **Editable** in eine eigene Datei (in Eclipse über File → New → Interface). Es besitzt die (öffentlichen) **void**-Methoden

- **addPrefix(String s)**, und
- **addPostfix(String s)**.

Schreiben Sie eine Klasse namens **String2**, die **Editable** implementiert.

String2 besitzt als Objekt-Attribut ein **String**-Attribut namens **mainString**.

Als Konstruktor gibt es **String2(String s)**, der den in **s** gespeicherten Wert an **mainString** zuweist.

String2 implementiert außerdem die Methode **addPrefix(String s)** so, dass vor dem in **mainString** enthaltenen Text noch der übergebene **String** vorgelagert wird. Analoges gilt für **addPostfix(String s)**, nur dass hier dann **s** nachgelagert wird.

- Ist in **mainString** das Wort **abc** gespeichert, dann bewirkt **addPrefix("!")**, dass in **mainString** dann **!abc** gespeichert ist. Bei **addPostfix("!")** wird dementsprechend dann **abc!** gespeichert sein.

Implementieren Sie schließlich die aus Vorlesung und Übung bekannte **toString**-Methode für **String2**, so dass beim Aufruf der **print**-Methode der in **mainString** gespeicherte Wert angezeigt wird.

Erstellen Sie im Repository ein Package **h1** und fügen Sie diesem Package die Klasse **H1_main** hinzu, welche die **main**-Methode enthält. Conway's **Game of Life** ist ein berühmtes zelluläres Automatenmodell, das der Mathematiker John Horton Conway 1970 entwickelte. Es handelt sich um ein einfaches Modell, das eine Art "Spiel" auf einem Gitter darstellt, bei dem sich Zellen in Abhängigkeit von ihren Nachbarn entwickeln. Es wird oft verwendet, um komplexe Systeme mit einfachen Regeln zu simulieren. Siehe <https://playgameoflife.com>.

In dieser Aufgabe wollen wir eine einfache Version davon erstellen.

Regeln des Spiels

Das Spielfeld ist ein unendliches (oder oft begrenztes) zweidimensionales Gitter, auf dem jede Zelle entweder **lebendig** (1) oder **tot** (0) ist. Das Spiel verläuft in **Iterationen**, und in jedem Schritt wird der Zustand jeder Zelle gemäß der folgenden Regeln aktualisiert:

1. **Überbevölkerung:** Eine lebende Zelle mit mehr als 3 lebenden Nachbarn stirbt.
2. **Einsamkeit:** Eine lebende Zelle mit weniger als 2 lebenden Nachbarn stirbt.
3. **Fortpflanzung:** Eine tote Zelle mit genau 3 lebenden Nachbarn wird lebendig.
4. **Überleben:** Eine lebende Zelle mit 2 oder 3 lebenden Nachbarn bleibt am Leben.

Die Nachbarschaft umfasst die acht Zellen, die eine Zelle direkt umgeben (oben, unten, links, rechts und diagonal).

Ablauf des Spiels

1. **Initialisierung:** Ein Startmuster wird definiert, indem einige Zellen lebendig gesetzt werden.
2. **Iterationen:** Die obigen Regeln werden gleichzeitig auf alle Zellen angewandt, um die nächste Generation zu bestimmen.
3. Das Spiel endet entweder, wenn:
 - sich das Gitter nicht mehr verändert (stabile Konfiguration),
 - alle Zellen tot sind,
 - oder nach einer gewünschten Anzahl von Schritten.

Die Klasse Grid

Schreiben Sie die Klasse **Grid**. Diese Klasse hat als Objektattribut

- das zweidimensionale **Cell**-Array **gridArray**.

Sie dürfen davon ausgehen, dass jede Zeile in **gridArray** die gleiche Anzahl an Spalten besitzt.

Die Klasse besitzt für das Objektattribut Getter und Setter.

Die Klasse besitzt den Konstruktor **Grid(Cell[] cells, int gridRows, int gridCols)**. In diesem Konstruktor wird das **gridArray** mit **gridRows** vielen Zeilen und **gridCols** vielen Spalten erzeugt. Weiterhin müssen für alle Einträge in **gridArray** Zellen mit den dazugehörigen Indizes erzeugt werden. Weiterhin werden alle die Zellen als lebendig markiert, deren Indizes mit den Indizes in den Zellen in **cells** übereinstimmt. Dabei spielt es keine Rolle, welchen Status die **Cell**-Objekte in **cells** haben. Falls die Indizes der **Cell**-Objekte in **cells** Indizes haben, welcher keiner Zelle im **gridArray** entsprechen, so wird aufgrund dieses **Cell**-Objekts aus **cells** kein **Cell**-Objekt in **Grid** als lebendig markiert (siehe Beispiel). Anschließend werden mittels Aufruf der Methode **countLivingNeighbors** die Attribute **numLivingNeighbors** und **isAliveNextGen** in allen Zellen korrekt gesetzt.

- Beispiel: **cells** enthält zwei **Cell**-Objekte. Das erste **Cell**-Objekt hat die Indizes (0,1), das zweite hat die Indizes (4,5). Dann müssen die in **Grid** enthaltenen Zellen mit Indizes (0,1) und (4,5) als lebendig gekennzeichnet werden. Enthält **Grid** nur drei Zeilen, so wird nur die Zelle mit Indizes (0,1) als lebendig markiert.

Die Methode **computeNextGen** aus **Grid**

Schreiben Sie die Methode **computeNextGen**.

- Modifikatoren: **public**
- Rückgabotyp: **void**
- Name: **computeNextGen**
- Übergabeparameter: keine

Diese Methode errechnet ausgehend von **gridArray** nach den vorgegebenen Regeln das Grid der nächsten Generation und speichert die entsprechenden Werte in **gridArray**. Das bedeutet explizit, dass die Anzahl der lebenden Nachbarn einer Zelle und ob die Zelle lebt oder stirbt aktualisiert werden muss.

Die Methode **computeGeneration** aus **Grid**

Schreiben Sie die Methode **computeGeneration**.

- Modifikatoren: **public**
- Rückgabotyp: **void**
- Name: **computeGeneration**
- Übergabeparamter: **int n**

Die Methode berechnet ausgehend von **gridArray** die **n**-te Generation und speichert die **n**-te Generation in **gridArray**. Sie dürfen davon ausgehen, dass

- `n` nicht-negativ ist,
- `n` gleich 0 bedeutet, dass **gridArray** unverändert bleibt.

Beispiel:

- Im folgenden Beispiel symbolisiert die 1 (bzw. 0) eine lebendige (bzw. tote) Zelle.
Sei **gridArray** wie folgt belegt:

```
0 0 0 0 1
0 1 0 0 0
0 0 1 1 0
0 1 1 0 0
```

computeGeneration(1) liefert das gleiche wie **computeNextGen()**:

```
0 0 0 0 0
0 0 1 1 0
0 0 0 1 0
0 1 1 1 0
```

computeGeneration(2) liefert:

```
0 0 0 0 0
0 0 1 1 0
0 1 0 0 1
0 0 1 1 0
```

Die Klasse Cell

Schreiben Sie die Klasse **Cell**. Diese Klasse hat als Objektattribute

- das **int**-Attribut **indexRow**, das dem Zeilenindex der Zelle in **gridArray** entspricht,
- das **int**-Attribut **indexCol**, das dem Spaltenindex der Zelle in **gridArray** entspricht,
- das **boolean**-Attribut **alive**, das dem Status der Zelle entspricht: **true** bedeutet, dass die Zelle lebendig ist, **false**, dass sie tot ist.
- das **int**-Attribut **numLivingNeighbors**, welches die Anzahl der lebenden Nachbarn in der aktuellen Generation speichert,
- das **boolean**-Attribut **isAliveNextGen**, welches speichert, ob die Zelle in der nächsten Generation lebendig oder tot sein wird.

Die Klasse besitzt die Getter und Setter zu allen Attributen.

Die Klasse **Cell** besitzt zwei Konstruktoren.

- Den Konstruktor **Cell(int indexRow, int indexCol, boolean alive)**, welcher die entsprechenden Werte an die Attribute von **Cell** zuweist.
- Den Konstruktor **Cell(int indexRow, int indexCol)**, welcher die entsprechenden Werte an die Attribute von **Cell** zuweist.

Cell besitzt die Methode **countLivingNeighbors**.

- Modifikatoren: **public**
- Rückgabetyt: **void**
- Name: **countLivingNeighbors**
- Übergabeparameter: **zweidimensionales Cell-Array gridArray**

Diese Methode berechnet die Anzahl der lebendigen Nachbarn des aufrufenden Zellobjekts und setzt dementsprechend direkt das Attribut **numLivingNeighbors** auf den richtigen Wert. Dabei gelten als Nachbarn alle die Zellen, die sich mit der aktuellen Zelle in mindestens einem Punkt schneiden und nicht die Zelle selbst sind. Eine Zelle hat folglich 8 Nachbarn. Anschließend ruft diese Methode noch **decideNextStatus** auf, sodass auch das Attribut **isAliveNextGen** den richtigen Wert zugewiesen bekommt (siehe Methodenbeschreibung **decideNextStatus**).

Cell besitzt die Methode **decideNextStatus**.

- Modifikatoren: **private**
- Rückgabetyt: **void**
- Name: **decideNextStatus**
- Übergabeparameter: keine

Diese Methode entscheidet, ob die Zelle in der nächsten Generation lebendig oder tot sein wird und setzt das entsprechende Attribut **isAliveNextGen** (**true** bedeutet: ist in der nächsten Generation lebendig, **false** bedeutet: ist in der nächsten Generation tot). Dabei gilt: Ist die Zelle lebendig und hat zwei oder drei lebendige Nachbarn, so bleibt sie lebendig. Bei weniger oder mehr Nachbarn stirbt die Zelle. Ist die Zelle tot, so erwacht sie in der nächsten Generation zum Leben, wenn genau drei Nachbarn lebendig sind. In jedem anderen Fall bleibt sie tot. Die Methode **decideNextStatus** wird im Rahmen der Methode **countLivingNeighbors** aufgerufen.

Hinweis:

- Wir weisen Sie darauf hin, dass - wie üblich - im Rahmen dieser Aufgabe keine Ausgabe stattfinden darf, die nicht direkt aus der **main**-Methode heraus verursacht wird.

Erstellen Sie im Repository ein Package **h2** und fügen Sie diesem Package die Klasse **H2_main** hinzu, welche die **main**-Methode enthält.

Sie sollen ein „Spiel“ implementieren, welches auf einem quadratischen Spielbrett gespielt wird, auf dem genau ein Spielstein steht. Dieser Spielstein startet auf einem beliebigen Feld des Spielbretts, und bewegt sich dann in Abhängigkeit der Richtung, die in dem Feld, auf dem er gerade steht, gespeichert ist.

Die Klasse **Spielbrett**

Schreiben Sie die Klasse **Spielbrett**. **Spielbrett** hat als Objekt-Attribute

- ein 2-dimensionales **Feld**-Array namens **brett**,
- ein **int**-Attribut **dim**.

Dabei hat **brett dim** viele Zeilen und **dim** viele Spalten. **Spielbrett** besitzt zudem die Getter und Setter zu **brett** und **dim**.

Konstruktor für **Spielbrett**

Fügen Sie den Konstruktor **Spielbrett(int dim)** hinzu. Dieser weist dem Objekt-Attribut **dim** den übergebenen Wert **dim** zu. Anschließend erzeugt er ein leeres 2-dimensionales **Feld**-Array mit **dim** Zeilen und **dim** Spalten und weist es dem Objektattribut **brett** zu.

Die Klasse **Feld**

Schreiben Sie die Klasse **Feld**. **Feld** besitzt als Objekt-Attribute

- das **boolean**-Attribute **boese**,
- das **char**-Attribut **direction**.

Dazu werden alle passenden Getter und Setter automatisch generiert. Der Wert **direction** steht für die Richtung, in welche vom Feld aus gelaufen werden muss. Das Attribut **direction** kann vier Werte annehmen:

- **U** für up (hoch)
- **D** für down (runter)
- **L** für left (links)
- **R** für right (rechts)

Sie dürfen also davon ausgehen, dass wir nur mit diesen vier Werten testen werden.

Weiterhin besitzt **Feld** den Konstruktor **Feld(boolean boese, char direction)**, welcher den entsprechenden Objekt-Attributen die übergebenen Werte zuweist.

Die Klasse **Spielstein**

Schreiben Sie die Klasse **Spielstein**. **Spielstein** besitzt als Attribute

- das **int**-Attribut **currentRow**,
- das **int**-Attribut **currentCol**,
- das **Spielbrett**-Attribut **brett**,

und die dazugehörigen Getter und Setter. Mit **currentRow** (bzw. **currentCol**) wird die Zeile (bzw. Spalte) bezeichnet, auf der ein Spielstein in **brett** steht.

Konstruktor für **Spielstein**

Fügen Sie den Konstruktor **Spielstein(Spielbrett brett, int indexRow, int indexCol)** hinzu. In diesem Konstruktor wird der Wert von **indexRow** an **currentRow** und **indexCol** an **currentCol** zugewiesen, sowie der Wert von **brett** an das Objekt-Attribut **brett** zugewiesen. Sie dürfen davon ausgehen, dass die Werte von **indexRow** und **indexCol** stets so gewählt sind, dass es auch ein Feld in **brett** mit diesen Indizes gibt.

Die Methode **movesOut**

Schreiben Sie die Methode **movesOut()** in die Klasse **Spielstein**.

- Modifikator: **private**
- Rückgabetyt: **boolean**
- Name: **movesOut**
- Übergabeparameter: keine

Die Methode **movesOut** gibt **true** zurück, wenn der Spielstein auf einem Feld steht, der als Wert von **direction** etwas gespeichert hat, das dazu führt, dass der Spielstein das Spielbrett verlässt.

- Beispiel: Betrachten Sie das unten angegebene Beispiel. Ist die aktuelle Position des Spielsteins das Feld mit **Indices (0,0)**, dann liefert der Aufruf von **movesOut** den Wert **true**, da ein **U** gespeichert ist.

Die Methode **go**

Spielstein besitzt weiterhin die Methode **go**.

- Modifikatoren: **public**
- Rückgabetyt: **void**
- Name: **go**
- Übergabeparameter: **int n**

Sie dürfen davon ausgehen, dass der in **n** gespeicherte Wert nicht-negativ ist. Wird **go(n)** aufgerufen, so vollführt der Spielstein **n** Züge auf dem Spielbrett **spielbrett** nach den vorgegebenen Regeln. Bei jedem Zug werden dabei die Werte von **currentRow** und **currentCol** aktualisiert, so dass die aktuelle Position des Spielsteins auf dem Brett stets nachvollzogen werden kann. Ist **n** gleich 0, so bleiben **currentRow** und **currentCol** unverändert.

Ein einzelner Zug wird nach den folgenden Regeln vollzogen: Speichert das Feld, auf dem sich der Spielstein gerade befindet, für **boese** den Wert **false** und für **direction** ein

- **U**, so macht der Spielstein einen Schritt nach oben, wechselt also in das Feld unmittelbar über dem Feld, auf dem der Spielstein sich gerade befindet.
- **D**, so macht der Spielstein einen Schritt nach unten, wechselt also in das Feld unmittelbar unter dem Feld, auf dem der Spielstein sich gerade befindet.
- **L**, so macht der Spielstein einen Schritt nach links, wechselt also in das Feld unmittelbar links neben dem Feld, auf dem der Spielstein sich gerade befindet.
- **R**, so macht der Spielstein einen Schritt nach rechts, wechselt also in das Feld unmittelbar rechts neben dem Feld, auf dem der Spielstein sich gerade befindet.

Speichert das Feld, auf dem sich der Spielstein gerade befindet, für **boese** den Wert **true**, so verbleibt der Spielstein auf seiner aktuellen Position. Sollte der Spielstein aufgrund obiger Regeln auf ein Feld ziehen müssen, das nicht existiert, so besteht der daraus resultierende Zug darin, dass der Spielstein auf seiner aktuellen Position verbleibt.

- Beispiel: Speichert in dem unteren Beispiel das Feld mit den Indizes (2,0) für **boese** den Wert **true**, und für alle anderen Felder ist **false** der Wert in **boese** gespeichert. So bewegt sich der Spielstein nach dem dritten Schritt nicht mehr weiter.

Beispiel 1:

Stellen Sie sich zum Beispiel ein 5×5 -Spielbrett vor, in dessen Felder die folgenden Richtungen gespeichert sind, und in dem alle Felder für **boese** den Wert **false** gespeichert haben:

```
U R L L D
L R L L D
U L L L R
D R U L L
U R L D D
```

Der Spielstein steht beispielsweise auf der Zelle mit Indizes (3,2). Hier ist ein **U** gespeichert. Wird nun **go(6)** aufgerufen, dann steht der Spielstein nach dem

1. Schritt auf dem Feld mit Indizes (2,2),
2. Schritt auf dem Feld mit Indizes (2,1),
3. Schritt auf dem Feld mit Indizes (2,0),
4. Schritt auf dem Feld mit Indizes (1,0).

Bei den weiteren Schritten bleibt der Spielstein stets auf dem Feld mit Indizes (1,0) stehen.

Beispiel 2:

In einem 3×3 - Spielbrett sind folgende Richtungen gespeichert:

L R L

L R U

L U U

Weiterhin sind folgende Werte für boese gespeichert:

false true false

false true false

false false false

Angenommen, der Spielstein startet auf dem Feld mit Indizes (2,2). Wird nun `go(7)` aufgerufen, dann zieht der Spielstein auf (1,2), dann auf (0,2), dann (0,1) und bleibt dann da stehen, weil boese hier den Wert `true` speichert.

- Hinweis: Entfernen Sie alle Testausgaben vor Ihrer Abgabe.